

```
from google.colab import files
files.upload()
```

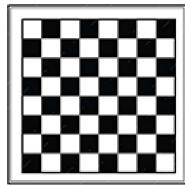
[Show hidden output](#)

```
import os
import cv2 as cv
import numpy as np
import matplotlib.pyplot as plt
from IPython.display import Image
%matplotlib inline

# =====
# DISPLAY IMAGES
# =====
display(Image(filename="/content/chessboard18X18.png"))
```



```
display(Image(filename="/content/chessboard84X84.jpg"))
```



```
# Reading image as gray scale.
path = r"/content/chessboard18X18.png"
img = cv.imread(path, cv.IMREAD_GRAYSCALE)
# Print the image data (pixel values), element of a 2D numpy array. Each pixel value is 8-bit [0,255]
print(img)
```

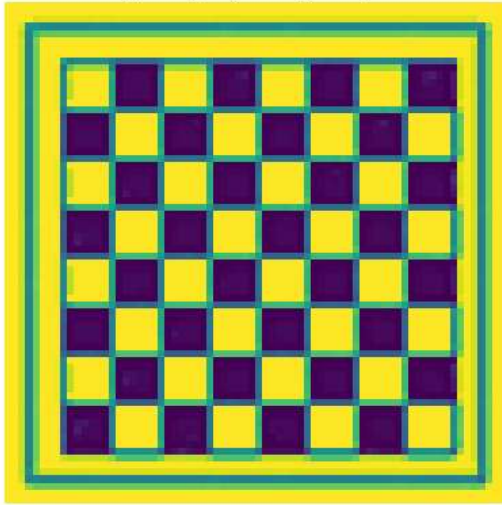
```
[[255 255 255 ... 255 255 255]
 [255 255 255 ... 255 255 255]
 [255 255 255 ... 255 255 255]
 ...
 [255 255 255 ... 252 255 255]
 [255 255 255 ... 255 255 255]
 [255 255 255 ... 255 255 255]]
```

```
# Displaying the image attributes
print("Shape of the image: ", img.shape)
print("Data type of the image: ", img.dtype)
print("Size of the image: ", img.size)
```

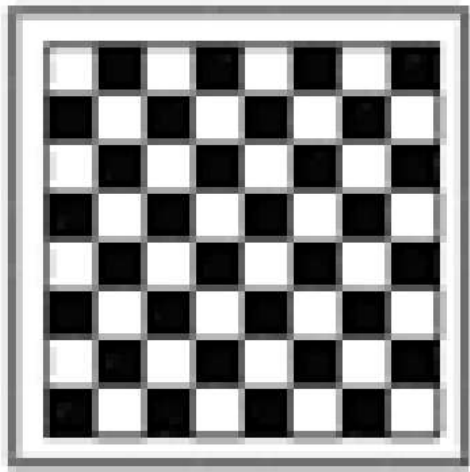
```
Shape of the image: (72, 72)
Data type of the image: uint8
Size of the image: 5184
```

```
# Displaying an image using Matplotlib.
plt.imshow(img)
plt.title("Image Display Using Matplotlib")
plt.axis("off") # To turn off axes
plt.show()
```

Image Display Using Matplotlib



```
# Set color map to gray scale for proper rendering.  
plt.imshow(img, cmap = "gray")  
plt.axis("off")  
plt.show()
```



```
path2 = r"/content/aura.jpg"  
Image(filename = path2)
```



```
#Reading image in BGR
img = cv.imread(path2, cv.IMREAD_COLOR_BGR)
print("Shape of the image: ", img.shape)
print("Data type of the image: ", img.dtype)
print("Size of the image: ", img.size)
```

```
Shape of the image: (576, 576, 3)
Data type of the image: uint8
Size of the image: 995328
```

```
#Display the image using matplotlib
plt.imshow(img)
plt.axis("off")
plt.show()
```



```
#Reversing the channels
reversed_img = img[:, :, ::-1]
plt.imshow(reversed_img)
plt.axis("off")
plt.show()
```



```
# Split the image into the B, G, R components
b, g, r = cv.split(img)
# Show the channels
plt.figure(figsize = [20, 5])
# Red Channel
plt.subplot(1, 4, 1)
plt.imshow(r, cmap = "gray")
plt.title("Red Channel")
plt.axis("off")
# Green Channel
plt.subplot(1, 4, 2)
plt.imshow(g, cmap = "gray")
plt.title("Green Channel")
plt.axis("off")
# Blue Channel
plt.subplot(1, 4, 3)
plt.imshow(b, cmap = "gray")
plt.title("Blue Channel")
plt.axis("off")
# Merge the individual channels into a BGR image
merged_img = cv.merge((b, g, r))
# Show the merged output
plt.subplot(1, 4, 4)
plt.imshow(merged_img[:, :, ::-1])
plt.title("Merged Output")
plt.axis("off")
plt.show()
```



```
# OpenCV stores color channels in a different order than most other applications
img_bgr = cv.imread(path2, cv.IMREAD_COLOR_BGR)
# Converting BGR to RGB
img_rgb = cv.cvtColor(img_bgr, cv.COLOR_BGR2RGB)
plt.imshow(img_rgb)
plt.title("BGR to RGB")
plt.axis("off")
plt.show()
```


BGR to RGB



```
# Converting to HSV
img_hsv = cv.cvtColor(img_bgr, cv.COLOR_BGR2HSV)
# Split the image into H, S, V components
h, s, v = cv.split(img_hsv)
# Show the channels
plt.figure(figsize = [20, 5])
# Hue Channel
plt.subplot(1, 4, 1)
plt.imshow(h, cmap = "gray")
plt.title("H Channel")
plt.axis("off")
# Saturation Channel
plt.subplot(1, 4, 2)
plt.imshow(s, cmap = "gray")
plt.title("S Channel")
plt.axis("off")
# Value Channel
plt.subplot(1, 4, 3)
plt.imshow(v, cmap = "gray")
plt.title("V Channel")
plt.axis("off")
# Show the original image
plt.subplot(1, 4, 4)
plt.imshow(img_rgb)
plt.title("Original")
plt.axis("off")
plt.show()
```

H Channel



S Channel



V Channel



Original



```
img_gray = cv.cvtColor(img_bgr, cv.COLOR_BGR2GRAY)
plt.imshow(img_gray, cmap = "gray")
plt.title("BGR to Grayscale")
plt.axis("off")
plt.show()
```

BGR to Grayscale



```
h_new = h + 30
img_merged = cv.merge((h_new, s, v))
img_rgb = cv.cvtColor(img_merged, cv.COLOR_HSV2RGB)
# Split the image into H, S, V components
h, s, v = cv.split(img_merged)
# Show the channels
plt.figure(figsize = [20, 5])
# Hue Channel
plt.subplot(1, 4, 1)
plt.imshow(h, cmap = "gray")
plt.title("H + 30 Channel")
plt.axis("off")
# Saturation Channel
plt.subplot(1, 4, 2)
plt.imshow(s, cmap = "gray")
plt.title("S Channel")
plt.axis("off")
# Value Channel
plt.subplot(1, 4, 3)
plt.imshow(v, cmap = "gray")
plt.title("V Channel")
plt.axis("off")
# Show the original image
plt.subplot(1, 4, 4)
plt.imshow(img_rgb)
plt.title("New Merged Image")
plt.axis("off")
plt.show()
```

H + 30 Channel



S Channel



V Channel



New Merged Image



```
# Saving the image
cv.imwrite(r"/content/aura_rgb.jpg",img_rgb)
```

```
True
```